Agrupamento de estilos de direção em um veículo

usando dados de acelerômetro (telemetria veicular)

Tópicos Avançados em Inteligência Artificial

Prof. Dr. Márcio Andrey Teixeira

Integrantes:

Mikael Guilherme Zana

Gabriella Cespedes

Link da aplicação: preencher após o deploy

1. Identificação

Título do projeto: Agrupamento de estilos de direção em um veículo usando dados de acelerômetro.

Integrantes: Mikael Guilherme Zana e Gabriella Cespedes.

2. Problema e objetivo

Os veículos de uma frota recebem um aparelho de telemetria (IMU + GPS) que, a cada 15 segundos, aproximadamente, registra a dinâmica de direção: velocidade, inclinação frontal (*pitch*), rolagem lateral (*roll*) e aclive ou declive do terreno (*slope*). O objetivo do trabalho é descobrir, sem o uso de rótulos, quais perfis de direção existem nesses dados — ou seja, agrupar os *reports* em padrões de condução semelhantes (por exemplo, manobra em baixa velocidade, trecho de pista, subida ou descida) e interpretar o que cada grupo representa.

Trata-se de um problema não supervisionado: não há uma resposta correta por *report*. A técnica utilizada é o agrupamento (K-means), e a questão de investigação é quantos perfis distintos de direção os dados sustentam e o que caracteriza cada um deles.

Público-alvo. Gestores de frota ou proprietários de veículos que desejam compreender como o veículo é conduzido — quanto tempo em manobra, em pista ou em aclive — sem precisar rotular os dados manualmente.

Entradas. Quatro valores de um *report* de telemetria: velocidade, *pitch*, *roll* e *slope*.

Resultado produzido. O perfil de direção (*cluster*) ao qual o *report* mais se assemelha, acompanhado do centro (perfil médio) do grupo. O resultado ajuda a caracterizar o contexto de direção daquele instante.

3. Conjunto de dados

Origem. Os dados foram obtidos de um veículo monitorado em um contexto profissional de rastreamento de frota, por meio de um aparelho de telemetria (IMU + GPS) instalado no automóvel. O aparelho grava um *report* a cada 15 segundos, aproximadamente, e registra originalmente um conjunto amplo de informações do veículo. Por questões de privacidade, boa parte dessas *features* originais foi removida antes do uso neste trabalho — foram mantidas apenas quatro variáveis numéricas que descrevem a dinâmica de direção, sem identificar o condutor, o itinerário detalhado ou qualquer outro dado sensível. O conjunto de dados chega, assim, no formato tabular adotado no projeto (`dados/telemetria.csv`), com as quatro *features* e as coordenadas (latitude e longitude) de cada *report*, usadas apenas na visualização em mapa.

Natureza não supervisionada. Não utilizamos nenhuma variável-alvo. O agrupamento parte somente dos quatro sinais de direção; os nomes atribuídos aos grupos são uma interpretação do perfil médio de cada *cluster*, e não rótulos oficiais presentes nos dados.

Atributos.

campo	unidade	significado
vel	km/h	velocidade medida pelo GPS
pitch	graus	inclinação frontal (acelera ou freia)
roll	graus	rolagem lateral (curvas)
slope	graus	aclive ou declive do terreno
lat/lon	graus	posição do *report* (apenas para o mapa; não entra no modelo)

Tamanho. 1914 *reports*. As quatro *features* são numéricas; não há variável-alvo.

Limitações do conjunto de dados. (i) Um *report* isolado contém cerca de 15 segundos de informação — descreve um instante de direção, e não o motorista; (ii) os sinais são dominados pela velocidade e pelo aclive/declive, de modo que o estilo fino de pilotagem (curvas) tem peso menor no agrupamento; (iii) os

dados provêm de um único veículo e de poucos trajetos, portanto os perfis encontrados refletem as rotas percorridas, com predomínio de uso urbano e pouco uso de pista. Essas limitações são retomadas na análise dos resultados.

4. Metodologia

4.1. Preparação dos dados

A preparação (`analise_preparacao.py`) parte do CSV de telemetria e executa as seguintes etapas:

- análise inicial: tipos das colunas e estatísticas (mínimo, máximo, média e desvio padrão) das quatro *features*;
- checagem de qualidade: contagem de valores ausentes (não há) e de linhas duplicadas;
- tratamento: remoção de valores nulos (não havia nenhum). As duplicatas são mantidas de forma intencional — no agrupamento, cada *report* é um ponto de direção válido, e *reports* repetidos (por exemplo, o veículo parado) são informação real sobre a frequência daquele estado;
- gráficos exploratórios (histograma das quatro *features*), salvos em `figuras/`.

Não há imputação nem codificação de categorias, pois as quatro *features* são numéricas e não existem valores ausentes. A normalização (StandardScaler) não é feita nesta etapa: ela é aprendida no treino, dentro de um *pipeline*, de modo a acompanhar o modelo no *deploy* e garantir que a inferência utilize a mesma escala do treino. O conjunto preparado é salvo em `dados/pronto.csv`.

4.2. Alternativas comparadas

A tarefa é o agrupamento com K-means. Como o número de grupos não é conhecido de antemão, a decisão de projeto consiste em definir quantos *clusters* utilizar. Comparamos duas alternativas:

- K-means com $k = 3$;
- K-means com $k = 4$.

Em ambas, as *features* são normalizadas (StandardScaler) antes do K-means, tudo dentro de um mesmo *pipeline*, com semente fixa (`random_state = 42`) e `n_init = 10` para garantir estabilidade do resultado.

4.3. Procedimento de avaliação

Como o problema é não supervisionado, não existem acurácia nem matriz de confusão. A qualidade do agrupamento é medida pelo coeficiente de silhueta médio: quanto mais próximo de 1, melhor a separação entre os grupos. Escolhemos o valor de k com maior silhueta e, em seguida, interpretamos o perfil médio de cada *cluster* para verificar se os grupos fazem sentido do ponto de vista da direção.

5. Resultados

alternativa	silhueta média
K-means, $k = 3$	0,3589
K-means, $k = 4$	0,3699

A silhueta de $k = 4$ (0,3699) é ligeiramente superior à de $k = 3$ (0,3589); portanto, $k = 4$ foi a alternativa escolhida para o *deploy*.



Figura 1: silhueta média por número de *clusters*

Perfis encontrados ($k = 4$). A partir da interpretação do centro (média de cada *feature*) dos quatro grupos:

cluster	perfil	vel	pitch	roll	slope	n
0	cidade/manobra	7,0	−2,0	−3,7	2,9	1045
1	pista (rápido)	85,4	−3,4	1,6	5,6	149
2	cidade em declive	14,4	3,7	−1,3	−7,0	387
3	cidade em aclive	11,6	−6,8	1,2	11,3	333

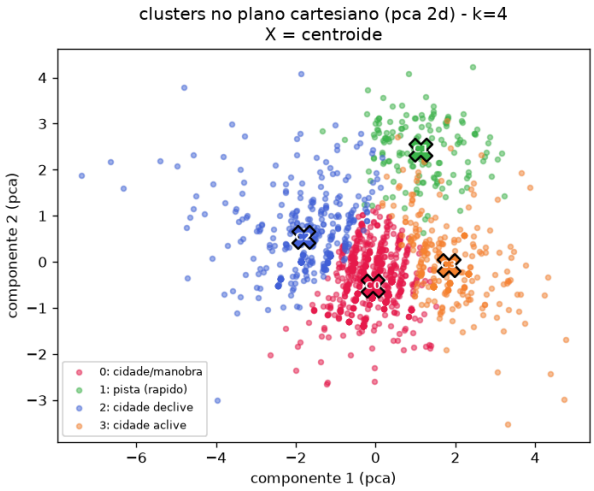


Figura 2: *clusters* no plano PCA ($k = 4$)

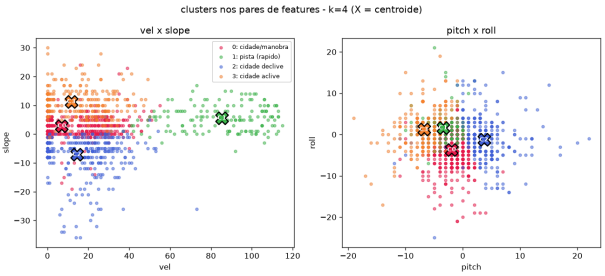


Figura 3: *clusters* nos pares de *features* ($k = 4$)

Leitura dos resultados. O agrupamento separa os *reports* principalmente pela velocidade e pelo aclive/declive. O *cluster* 1 isola o uso de pista (velocidade média de aproximadamente 85 km/h), bem destacado dos demais no plano PCA. Os outros três grupos são variações de uso urbano em baixa velocidade, diferenciados pelo terreno: o *cluster* 3 reúne a direção em aclive (*slope* elevado, +11,3), o *cluster* 2 corresponde à direção em declive (*slope* negativo, −7,0) e o *cluster* 0 representa a manobra em baixa velocidade e terreno quase plano, concentrando a maior parte dos *reports* (1045 de 1914). A silhueta moderada (cerca de 0,37) é coerente com o que se observa nos gráficos: há um grupo bem separado (pista) e três grupos urbanos que se tocam nas bordas.

Justificativa da escolha. Entre as duas alternativas, $k = 4$ apresentou a maior silhueta e, além disso, produziu uma interpretação mais rica — ao distinguir aclive de declive dentro do uso urbano — sem criar grupos minúsculos ou redundantes. Por esse motivo, o modelo com $k = 4$ foi re-treinado em todos os dados e salvo junto ao pré-processamento (`modelo/modelo_clusters.joblib`).

6. Aplicação e deploy

A aplicação foi desenvolvida em Streamlit (`app.py`) e é organizada em três abas.

A aba **classificar** apresenta a finalidade da ferramenta e permite inserir um *report* (velocidade, *pitch*, *roll* e *slope*); explica cada campo e sua unidade; valida as entradas, rejeitando campos vazios, valores não numéricos ou velocidade negativa, e emitindo um aviso quando algum valor está fora da faixa observada no treino; carrega o modelo salvo e executa uma inferência por consulta, sem re-treinar, atribuindo o *report* ao *cluster* mais próximo e exibindo o perfil e o centro do grupo.

A aba **mapa da rota** mostra o trajeto real do veículo (latitude e longitude), com cada *report* colorido de acordo com o seu *cluster*, o que torna visível se os perfis de direção formam regiões coerentes ao longo do percurso. O mapa utiliza apenas as coordenadas para o desenho; a atribuição do *cluster* continua baseada somente em velocidade, *pitch*, *roll* e *slope*.

A aba **plano cartesiano** projeta os *reports*, originalmente em quatro dimensões, em duas dimensões por meio de PCA, apenas para visualizar como os *clusters* se separam. São marcados os centroides e, caso o usuário já tenha classificado um ponto, o *report* informado por ele.

Link. *Preencher após a publicação no Streamlit Community Cloud.* As instruções de execução local e de *deploy* estão no README.

7. Testes e limitações

Casos de teste — os válidos conferidos em `casos_teste.py` e a entrada inválida verificada na própria aplicação:

tipo	**entrada**	**resultado**	**comentário**
válida 1	vel=10, pitch=−2, roll=−3, slope=2	cidade/manobra	baixa velocidade em terreno plano recai no grupo de manobra urbana
válida 2	vel=90, pitch=−3, roll=2, slope=5	pista (rápido)	velocidade alta é atribuída ao perfil de pista
válida 3	vel=12, pitch=−7, roll=1, slope=12	cidade em aclive	baixa velocidade com *slope* elevado recai no perfil de subida
inválida	campo vel vazio ou com texto	erro tratado	a aplicação exibe mensagem e não executa a atribuição
limítrofe	vel=115, pitch=22, roll=21, slope=30	*cluster* do perfil mais próximo	valores no limite da faixa observada; a aplicação emite aviso de baixa confiança

A mesma entrada produz o mesmo resultado tanto no código de inferência (`casos_teste.py`) quanto na aplicação, pois ambos carregam o modelo salvo. Como o problema é não supervisionado, não há um resultado real conhecido para uma entrada: a saída é o perfil mais próximo, e não uma verdade a respeito de quem conduz o veículo.

Limitações. (i) A decisão é tomada sobre um único *report* de cerca de 15 segundos — o *cluster* indica o contexto de direção mais provável daquele instante, e não a identidade do motorista; (ii) o agrupamento é dominado pela velocidade e pelo aclive/declive, de modo que o estilo fino de pilotagem tem peso menor; (iii) os *clusters* são perfis interpretados, e não categorias oficiais — os nomes refletem o perfil médio de cada grupo; (iv) a silhueta moderada (cerca de 0,37) indica que os grupos urbanos se tocam nas bordas, de forma que *reports* de fronteira podem recair em mais de um perfil plausível; (v) o volume de dados é pequeno, também em decorrência das restrições de privacidade descritas na seção do conjunto de dados — tanto pela redução das *features* quanto pela limitação da quantidade de registros disponíveis. O resultado é uma estimativa de apoio à interpretação, e não uma prova.

Como trabalho futuro, o estudo poderia ser reproduzido com um volume maior de dados e, se possível, com mais variáveis — provenientes de múltiplos veículos, condutores e trajetos. Espera-se que uma base mais ampla produza perfis de direção mais nítidos e estáveis, com silhueta mais elevada, além de permitir análises mais interessantes, como a comparação de estilos de condução entre diferentes motoristas ou veículos.

8. Contribuições e uso de IA

Contribuições. Mikael Guilherme Zana: definição do problema, do conjunto de dados, a engenharia das *features* e a revisão do *pipeline*. Gabriella Cespedes: avaliação dos modelos (silhueta e interpretação dos *clusters*), a aplicação em Streamlit, os testes e a redação do relatório. *(Ajustar conforme a divisão real do trabalho.)*

Uso de IA. O desenvolvimento foi apoiado por um assistente de IA de programação, utilizado para estruturar os *scripts* de preparação, treino e avaliação; montar a aplicação em Streamlit; e redigir rascunhos deste relatório. As decisões de modelagem — a seleção das *features*, a escolha do algoritmo de agrupamento, o número de *clusters* comparados e o critério de escolha do k — foram definidas pelo grupo. Todo o código e os resultados foram executados e revisados pelos integrantes antes da entrega.

Referências. scikit-learn (K-means, silhueta, PCA e *pipeline*); Streamlit e pydeck (aplicação web e mapa); pandas e NumPy (manipulação de dados); Matplotlib (gráficos). Conjunto de dados: coleta de telemetria de um veículo de teste.